

 dev

SGraph-AI__App__Send / team / roles / sherpa / reviews

Go to file

t

...

/ 26-02-23

/ v0.6.13__guide__what-is-sg-send.md



 **DinisCruz** adding 4 briefs for today (23 Feb) ●

22e42f5 · 1 minute ago



192 lines (114 loc) · 9.86 KB

Preview Code Blame

  Raw     

What is SG/Send?

Private file sharing that works like you'd expect — except nobody can read your files. Not even us.

The short version

SG/Send encrypts your files in your browser before they go anywhere. The server stores encrypted data it cannot read. The decryption key stays with you. You share it with whoever you choose.

No account. No app to install. Just a browser.

Try it at send.sgraph.ai

What you can do today

Send files — any size up to 1GB

Drop a file (or click to browse), hit **Encrypt & Upload**, and get a shareable link. The file is encrypted with AES-256-GCM in your browser before a single byte leaves your device.

Large files (up to 1GB) upload in parallel — 5 encrypted chunks at once — so a 42MB file takes about 15-20 seconds, not a minute.

Send text

Switch to text mode, type or paste whatever you need to share, and encrypt it. Same zero-knowledge encryption, same shareable link. Good for passwords, API keys, notes, or anything you don't want sitting in an email.

Share with a link (and optionally, a separate key)

After upload, you get a link that includes the decryption key in the URL fragment (the part after # — it never reaches the server).

For extra security, you can split them: send the link via email, send the key via Signal or WhatsApp. Two channels, one transfer. Even if someone intercepts the link, they can't decrypt without the key.

See exactly what we stored

After a download, SG/Send shows you a transparency panel:

What we stored	What we never stored
A hash of your IP address (not the actual IP)	Your original IP address
Upload timestamp	The file name (encrypted — we can't read it)
File size	The file content (encrypted — we can't read it)
Encryption method used	The decryption key (only you have it)

That's it. Nothing else is captured. We show you this because we think you should know.

Store files in your encrypted vault

Beyond one-off transfers, you can store files in a personal encrypted vault. Upload, browse, preview, edit, and share — all encrypted, all in your browser. Think of it as a private file manager where the server stores ciphertext it cannot read.

Why SG/Send?

Your files are encrypted before they leave your device

This is the fundamental difference between SG/Send and most file-sharing services.

When you use WeTransfer, Google Drive, or Dropbox, the service can read your files. They may promise not to. They may encrypt files "at rest" on their servers. But the encryption keys are theirs, not yours. If their servers are breached, or if they receive a legal request, your files are readable.

SG/Send works differently. The encryption happens in your browser using the Web Crypto API. The server receives encrypted data and stores it. It never has the key. It cannot decrypt your files even if someone gains full access to the infrastructure.

This is called **zero-knowledge encryption**. It means we know that you transferred something, but we cannot know what.

No account, no friction

You don't need to create an account. You don't need to give us your email. The sender gets a token, uploads, and shares a link. The recipient clicks the link, enters the key, and downloads. That's the whole flow.

This matters for recipients especially. When someone sends you a file via SG/Send, you don't need to sign up for anything. You just click and decrypt.

Open source

The code is public under the Apache 2.0 licence. You can read the encryption implementation, verify our claims, and — if you want — deploy SG/Send on your own infrastructure. The zero-knowledge promise is verifiable, not just a line in a privacy policy.

No special software required

Any modern browser with HTTPS. Chrome, Firefox, Safari, Edge. Desktop or mobile. No browser extension, no desktop app, no Tor. Just a URL.

How it works (the simple version)



```
graph TD; A[You select a file] --> B[Your browser encrypts it (AES-256-GCM)]; B --> C[Encrypted data uploads to the server]; C --> D[You get a link with the decryption key]; D --> E[You share the link with someone]; E --> F[Their browser downloads the encrypted data]; F --> G[Their browser decrypts it with the key]; G --> H[They have the original file];
```

The server's only job is storing encrypted data and handing it back. It never has the key. It never sees the original file. It never knows the file name.

For large files (over 5MB), the encrypted data is split into chunks and uploaded directly to storage in parallel — 5 chunks at a time. The server coordinates this but never touches the file data itself.

Use cases (things people actually do with SG/Send today)

Sharing sensitive documents

Contracts, tax returns, medical records, HR documents, financial statements. Anything where "I emailed it" makes your security team uneasy. SG/Send means the transfer service cannot read what was sent — and there's a transparency panel to prove it.

Sending credentials and secrets

API keys, passwords, SSH keys, configuration files with secrets. Instead of pasting them into Slack or email (where they're stored in plaintext, searchable, and visible to admins), send them via SG/Send. The recipient decrypts once; the server never had the plaintext.

Sharing files with people outside your organisation

When you need to send something to a client, a partner, or a contractor, you can't always control what tools they use. SG/Send doesn't require the recipient to have an account, install an app, or be on the same platform. They just need a browser.

Replacing "I'll just use WeTransfer"

For teams that care about privacy but don't want the friction of enterprise tools, SG/Send is the drop-in replacement. Same simplicity as WeTransfer, but zero-knowledge encrypted. Your IT team can verify the code. Your recipients don't need to know the difference.

Personal encrypted storage

The vault lets you keep files encrypted and accessible from any browser. Think of it as a personal safe deposit box — you hold the only key, and we store a locked box we cannot open.

What SG/Send is not (being honest)

It's not a replacement for Dropbox or Google Drive. It doesn't do real-time collaboration, folder syncing, or team workspaces. It's a transfer and storage tool, not a productivity suite.

It's not the largest file-sharing service. WeTransfer supports files up to 200GB on their paid plan. SG/Send currently supports up to 1GB. If you need to send a 50GB video file, we're not there yet.

It's not anonymous. We store a hash of your IP address and the upload timestamp. We can't see what you sent, but we know a transfer happened and roughly when and from where.

It's beta software. It works, it's tested (297 tests passing), and it's actively developed. But it's early. The UX is functional, not polished. If you're looking for a mature, enterprise-certified product with SLAs and phone support, that's not us yet.

It doesn't hold compliance certifications. We don't have SOC 2, ISO 27001, or HIPAA certification. If you self-host SG/Send, it runs under your compliance framework — but we can't hand you a certificate.

How SG/Send compares

	SG/Send	WeTransfer	Tresorit Send	Firefox Send
Zero-knowledge encryption	Yes	No	Yes	Was yes
Max file size	1 GB	2 GB (free) / 200 GB (Pro)	5 GB	Was 2.5 GB
Recipient needs account?	No	No	Yes	Was no
Open source	Yes	No	No	Was yes
Self-hostable	Yes	No	No	Was yes
Status	Active, beta	Active	Active	Shut down (2020)

We're not trying to be bigger than WeTransfer. We're trying to be the tool you use when privacy actually matters — and when you want to verify that claim by reading the code.

A note on how SG/Send is built

SG/Send is developed by a small team using an unusual approach: one human lead (Dinis Cruz) working with a team of 18 specialised AI agents. Each agent has a defined role — Architect, Developer, QA, Security, Designer, and others — operating under a structured methodology based on Wardley Maps.

The agents build, review, test, and document the product. The human sets direction, makes decisions, and validates the output. It's a real working team that produces real code, tests, security reviews, and documentation — all versioned and auditable in the Git history.

Here's the interesting part: while AI agents build SG/Send, there is zero AI in the production product. No AI processes your files. No AI makes decisions about your data. No AI runs when you upload, download, or decrypt. The product is deterministic, auditable code — standard Python backend, vanilla JavaScript frontend, Web Crypto API for encryption.

The AI is the workshop. The product is the furniture. You use the furniture; you never interact with the workshop.

This matters because it means:

- The encryption is standard, well-understood cryptography — not AI-generated decisions
- The code can be audited by any security researcher
- The product behaves predictably and deterministically
- There are no AI hallucination risks in your file transfer pipeline

We mention this because the "AI-built, no-AI-in-production" approach is something we think more projects should consider. The efficiency of AI-assisted development, combined with the reliability of deterministic production code, is a genuinely useful pattern.

Get started

1. Go to send.sgraph.ai
2. Drop a file or switch to text mode
3. Click **Encrypt & Upload**
4. Share the link

That's it. Your file is encrypted. We can't read it. Nobody can, except whoever has the link and the key.

This guide was written collaboratively by the Sherpa (user guidance), Ambassador (positioning), and Designer (accessible messaging) roles — 23 February 2026, v0.6.13